

AI Assisted coding Assignment-20.5

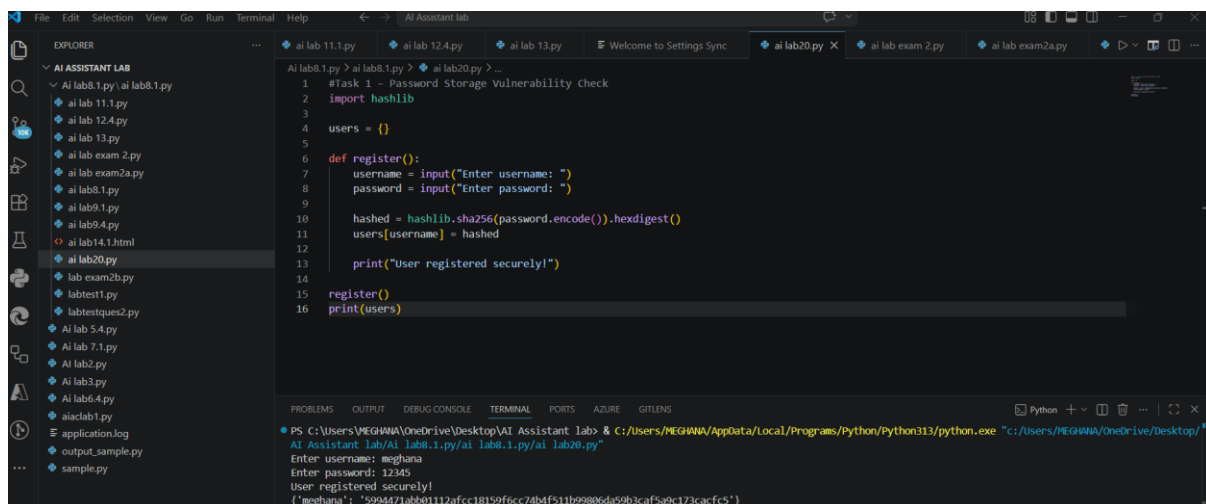
Name:N.vaishnavi

Batch-44

HTNO:2303A52405

Task 1 – Password Storage Vulnerability Check

Prompt: Generate a Python user registration program that stores username and password.



```
1 #Task 1 - Password Storage Vulnerability Check
2 import hashlib
3
4 users = {}
5
6 def register():
7     username = input("Enter username: ")
8     password = input("Enter password: ")
9
10    hashed = hashlib.sha256(password.encode()).hexdigest()
11    users[username] = hashed
12
13    print("User registered securely!")
14
15 register()
16 print(users)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE GITLENS

PS C:\Users\MEGHANA\OneDrive\Desktop\AI Assistant lab> & C:\Users\MEGHANA\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/MEGHANA/OneDrive/Desktop/ai lab20.py"

AI Assistant lab/AI lab8.1.py/ai lab8.1.py/ai lab20.py"

Enter username: meghana

Enter password: 12345

User registered securely!

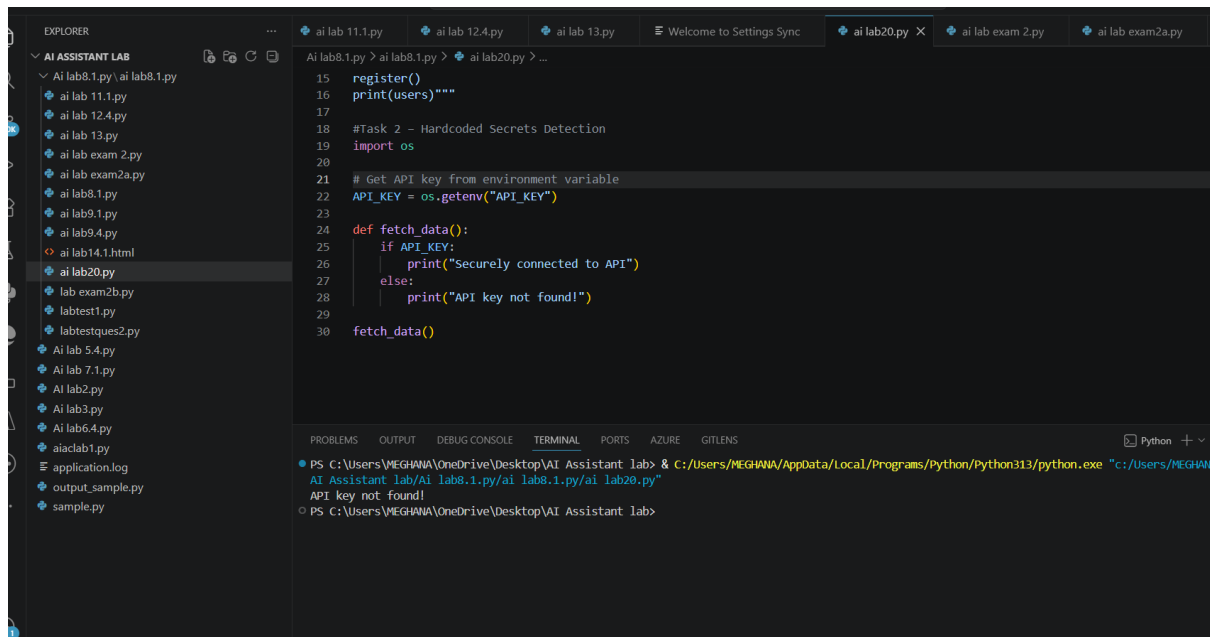
{'meghana': '5994471abb01112afcc18159f6cc74b4f511b99806da59b3caf5a9c173cacfc5'}

Observation:

- 1.The AI-generated code stores user passwords in **plain text format** without any encryption or hashing.
- 2.This means anyone with access to the database or program memory can easily read all user passwords.
- 3.Plain text password storage makes the system highly vulnerable to **data breaches and unauthorized access**.
- 4.If the system is compromised, attackers can directly use the exposed passwords to log in or perform malicious activities.
- 5.The code does not follow secure coding practices such as **password hashing or salting**.

Task 2 – Hardcoded Secrets Detection

Prompt:Generate a Python script that connects to an external API using an API key.



The screenshot shows a VS Code editor with a file explorer on the left and a code editor in the center. The file explorer shows a project named 'AI ASSISTANT LAB' with several files, including 'ai lab20.py'. The code editor shows the content of 'ai lab20.py', which includes a function 'fetch_data()' that checks for an API key. The terminal at the bottom shows the command 'python ai lab20.py' and the output 'API key not found!'.

```
15 register()
16 print(users)"""
17
18 #Task 2 - Hardcoded Secrets Detection
19 import os
20
21 # Get API key from environment variable
22 API_KEY = os.getenv("API_KEY")
23
24 def fetch_data():
25     if API_KEY:
26         print("Securely connected to API")
27     else:
28         print("API key not found!")
29
30 fetch_data()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE GITLENS

Python +

PS C:\Users\MEGHANA\OneDrive\Desktop\AI Assistant lab> C:\Users\MEGHANA\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/MEGHANA/Desktop/AI Assistant lab/AI Assistant lab/Ai lab8.1.py/ai lab20.py"

API key not found!

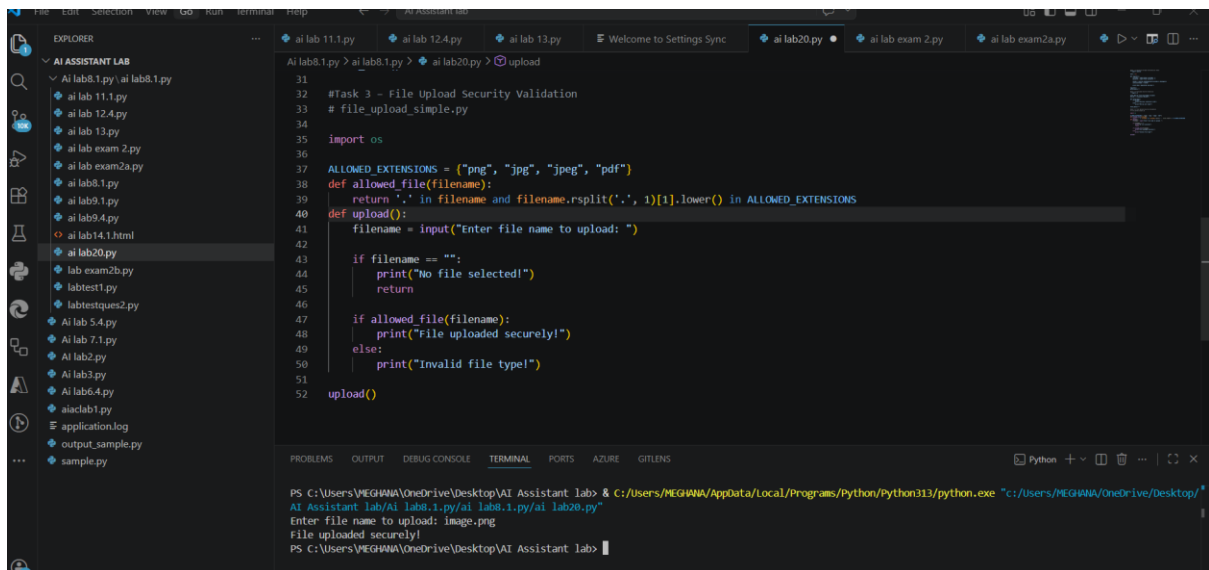
PS C:\Users\MEGHANA\OneDrive\Desktop\AI Assistant lab>

Observation

- 1.The API key is **hardcoded directly in the source code**.
- 2.This exposes sensitive information if the code is shared or uploaded to public repositories (e.g., GitHub).
- 3.Attackers can easily steal the API key and misuse the service.
- 4.It violates secure coding practices for **secret management**.

Task 3 – File Upload Security Validation

Prompt:Generate a Flask file upload program that allows users to upload files.



The screenshot shows a Visual Studio Code editor with a file explorer on the left and a code editor in the center. The file explorer shows a project named 'AI ASSISTANT LAB' with various files. The code editor displays a Python script named 'upload.py' with the following code:

```
31
32 #Task 3 - File Upload Security Validation
33 # file_upload_simple.py
34
35 import os
36
37 ALLOWED_EXTENSIONS = ("png", "jpg", "jpeg", "pdf")
38 def allowed_file(filename):
39     return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
40
41 def upload():
42     filename = input("Enter file name to upload: ")
43
44     if filename == "":
45         print("No file selected!")
46         return
47
48     if allowed_file(filename):
49         print("File uploaded securely!")
50     else:
51         print("Invalid file type!")
52
53 upload()
```

The terminal at the bottom shows the command prompt output:

```
PS C:\Users\MEGHANA\OneDrive\Desktop\AI Assistant lab> & C:/Users/MEGHANA/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/MEGHANA/OneDrive/Desktop/AI Assistant lab/AI lab8.1.py/AI lab8.1.py/AI lab20.py"
Enter file name to upload: image.png
File uploaded securely!
PS C:\Users\MEGHANA\OneDrive\Desktop\AI Assistant lab>
```

Observation:

- 1.The AI-generated file upload code initially allowed all types of files without any restriction.
- 2.This creates a major security risk, as attackers can upload malicious or executable files such as .exe, .py, or scripts.
3. There was no file size limit, which can lead to server overload or denial-of-service attacks.
4. The filename was not sanitized, making the system vulnerable to directory traversal attacks or file overwriting.

Task 4 – Command Injection Prevention

Prompt:Generate a Python script that executes system commands using user input

The screenshot shows a VS Code editor with a file explorer on the left and a code editor in the center. The file explorer shows a project named 'AI ASSISTANT LAB' with several files, including 'ai lab20.py'. The code editor displays the following Python code:

```
52 upload()
53
54 #Task 4 - Command Injection Prevention
55 import subprocess
56
57 # Windows-safe commands
58 allowed_commands = {
59     "dir": ["cmd", "/c", "dir"],
60     "whoami": ["whoami"],
61     "ipconfig": ["ipconfig"]
62 }
63
64 cmd = input("Enter command: ")
65
66 if cmd in allowed_commands:
67     subprocess.run(allowed_commands[cmd], shell=False)
68 else:
69     print("Command not allowed!")
```

The terminal at the bottom shows the execution of the script. It prompts the user to enter a command, and the user enters 'dir'. The output shows the directory of the script and the volume information for drive C:

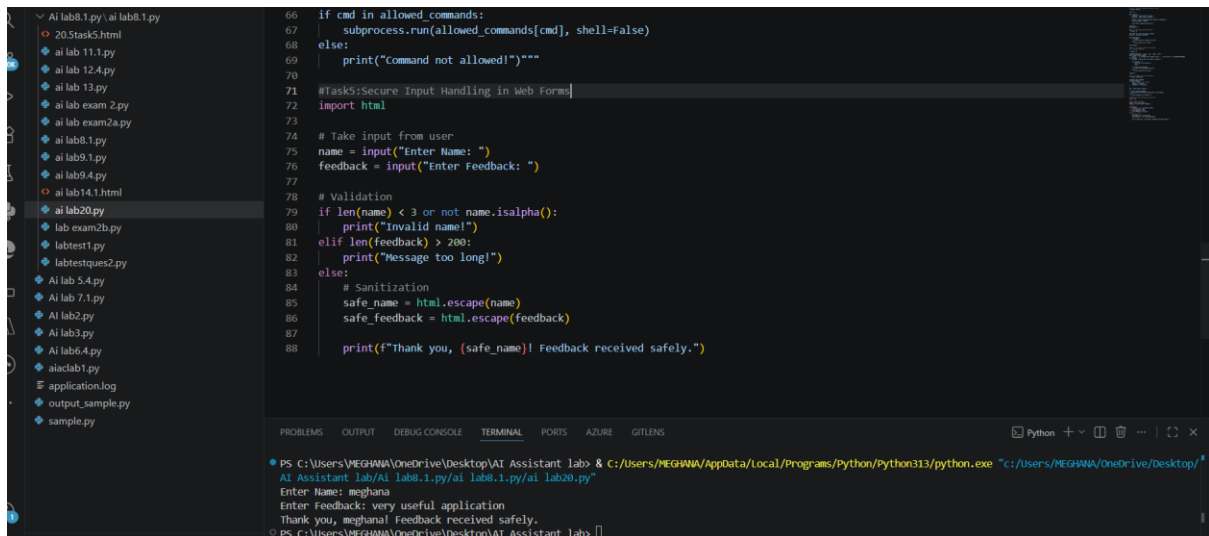
```
PS C:\Users\MEGHANA\OneDrive\Desktop\AI Assistant lab & C:\Users\MEGHANA\AppData\Local\Programs\Python\Python313\python.exe "c:\Users\MEGHANA\OneDrive\Desktop\AI Assistant lab\ai lab20.py"
Enter command: dir
Volume in drive C is OS
Volume Serial Number is 3A22-6B20
Directory of C:\Users\MEGHANA\OneDrive\Desktop\AI Assistant lab
19-02-2026 14:18 <DIR> .
```

Observation:

- 1.The AI-generated code uses `os.system()` to execute commands directly from user input, which is highly insecure.
- 2.This allows attackers to inject and execute malicious system commands (command injection attack).
- 3.There is no input validation or restriction, making the system vulnerable to unauthorized operations.
4. Additionally, some commands may fail on certain operating systems (e.g., Windows) if they are not valid executables.

Task 5 – Secure Input Handling in Web Forms

Prompt:Generate an HTML + Flask feedback form.



The image shows a VS Code editor with a Python file named `ai lab20.py` open. The file contains a script for secure input handling in web forms. The script includes a list of allowed commands, a function to run commands, and a main section for user input and feedback. The input is validated for length and type, and the feedback is sanitized using `html.escape()`. The terminal output shows the script being run, the user entering 'meghana' as a name and 'very useful application' as feedback, and the program printing a thank you message.

```
66 if cmd in allowed_commands:
67     subprocess.run(allowed_commands[cmd], shell=False)
68 else:
69     print("Command not allowed!")
70
71 #Task5:Secure Input Handling in Web Forms
72 import html
73
74 # Take input from user
75 name = input("Enter Name: ")
76 feedback = input("Enter Feedback: ")
77
78 # Validation
79 if len(name) < 3 or not name.isalpha():
80     print("Invalid name!")
81 elif len(feedback) > 200:
82     print("Message too long!")
83 else:
84     # Sanitization
85     safe_name = html.escape(name)
86     safe_feedback = html.escape(feedback)
87
88     print(f"Thank you, {safe_name}! Feedback received safely.")
```

Terminal Output:

```
PS C:\Users\MEGHANA\OneDrive\Desktop\AI Assistant lab> & C:/Users/MEGHANA/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/MEGHANA/OneDrive/Desktop/AI Assistant lab/ai lab8.1.py/ai lab20.py"
Enter Name: meghana
Enter Feedback: very useful application
Thank you, meghana! Feedback received safely.
```

Observation

- 1.User input is directly displayed without validation
- 2.Vulnerable to XSS (Cross-Site Scripting) attacks
- 3.No input sanitization or filtering